Spark Homes — Build Writeup

1 Most Interesting Design / UX Decision

Composite flat-map keying instead of nested data. The entire estimate is stored as a flat `cells` map keyed by `"roomId|groupId|itemId"`. Every room, group, and line-item interaction resolves to a single string lookup—no tree traversal, no recursive serialization. This was a counterintuitive choice for a hierarchical UI (rooms → groups → items), but it made the hardest operations trivial: deleting a room is a prefix sweep over `Object.keys`; computing a room total is a filter + reduce over matching keys; the entire state serializes to `localStorage` in one `JSON.stringify`. It also enabled the single `render()` function: since the data is flat, the view can be built in one pass with template literals, no component tree needed. The same flat model made snapshots and diffing nearly free (a snapshot is one deep copy; comparing versions is a set-difference over keys). The trade-off is a key scheme that's fragile to format changes, but for a contest-scoped app the simplicity won, and it eliminated a whole class of stale-reference bugs from deleted rooms and renumbered groups.

2 What Is Broken or Fragile

- **localStorage quota on large projects.** Photos are base64-encoded and stored inline. A thorough inspection with 50+ high-res photos can approach the ~5 MB localStorage limit on some browsers, causing silent save failures. The code compresses images to ≤1280 px and wraps writes with a quota check, but there is no graceful migration to IndexedDB—a production version would need that.
- **Full re-render on every mutation.** The `render()`-rebuilds-everything pattern is clean but causes scroll-position loss and input-focus jank on fast interactions (rapid quantity tapping, deal-tab typing). Targeted partial re-renders exist for the worst cases, but it's a band-aid; a virtual-DOM diff or keyed reconciliation would be the real fix.
- **AI model load time.** The Whisper + CLIP + MiniLM offline pack is ~90 MB total. First download on slow connections can time out or stall with no resume. The models are cached permanently after that, but there's no chunked/resumable download, and error recovery is limited to "try again."
- **Single-file scale.** At 3,650+ lines and 260 KB of JS/HTML/CSS in one file, `index.html` is nearing the limit of what's maintainable without a module system. IDE features like go-to-definition become slow; a production follow-up would split into ES modules behind a minimal bundler.

3 Creative Addition & Why

The Deal Analyzer + AI Copilot layer—built as a single creative system, not a cosmetic feature.

- **Deal Analyzer:** The brief says the repair estimate is the most important number—but an estimate in isolation doesn't answer the real question: *"Should I buy this house?"* The Deal Analyzer pulls the live repair total, adds ARV, purchase price, holding costs, and selling costs, then projects profit, ROI, and a 70%-rule max offer with a GO / CAUTION / NO-GO verdict. The agent gets a buy-or-walk decision *while still standing in the house*.
- **AI Copilot:** Four specialist modules—Inspection Coverage (flags missed big-ticket systems), Recommendations (commonly-paired work like "flooring → trim-out"), Anomaly & Risk (outlier quantities, price-override fraud, conflicting scope), and a semantic chat Assistant backed by on-device embeddings (MiniLM) over a repair-guidelines knowledge base. All 100% offline, no API key, no server. I chose this because the brief's multi-agent/RAG requirements shouldn't be faked with canned responses—this delivers the architecture and value on-device.
- **Voice Walkthrough + Camera Scope Detection:** Narrate repairs hands-free ("kitchen, cabinets, five doors"); on-device Whisper transcribes, a synonym+semantic matcher adds items with correct quantities. Point the camera at a fixture; CLIP classifies it and suggests the line item. Both work offline.
- **Field-trust layer:** because an agent is betting real money, I added a *contingency buffer* (slider or typed %, folded into the profit/verdict so a "GO" can't hide a thin margin), *estimate history with named snapshots* (compare any snapshot to now, item by item, to prove the scope wasn't inflated after a contractor re-walk), and timestamped, room-tagged *inspection notes* carried into the report and Excel. These turn a calculator into a tool an agent would actually trust.

4 What I'd Ship Next (Two More Days)

- **IndexedDB photo storage + multi-device cloud sync.** Migrate photos from base64-in-localStorage to IndexedDB blobs (removes the quota wall), then add optional Firebase/Supabase sync with conflict resolution so a team of agents can share estimates live. The flat-key data model makes merge logic straightforward—last-write-wins per cell key.
- **Comp-based ARV assistant.** Integrate a public MLS data source (Redfin/Zillow API or a scraped comps feed) so the Deal Analyzer can auto-suggest an ARV from recent sold comps instead of relying on manual input. Display comps on a map with price/sqft overlays and confidence bands. This closes the last manual gap in the deal math flow.

🤖 Role of AI Tools in Development

AI coding assistants (Claude, Gemini) acted as a **pair-programming partner** throughout: architecture brainstorming (the flat-key data model, the layered Copilot resolver, the offline-first service worker), prototyping boilerplate (the 108-item price map, the SVG charts, the semantic-search pipeline), debugging (re-render scroll bugs, iOS PWA quirks, Transformers.js WASM issues), and drafting the README, ARCHITECTURE, and this writeup. All AI-generated code was reviewed, tested, and iterated by hand: the AI accelerated implementation of decisions I made, it did not autonomously design features or make product calls.